

RTL Level Implementation of High Speed-Low Power Viterbi Encoder & Decoder

Pooran Singh and Santosh Kr. Vishvakarma

Abstract— High speed and low power Viterbi Encoder Decoder of rate $\frac{1}{2}$ convolutional coding with a constraint length $K = 3$ is presented in this paper. After implementation of proposed Viterbi encoder-decoder in Virtex 7 Field Programmable Gate Array (FPGA) kit we come to know that it's functioning on 393.544 MHz clock and in such a high speed it also maintain a low power of 11.34 mW in Spartan 6 FPGA. Since the FPGA boards used are different and from that we justified that using both logics together in one Integrated Circuit (IC) we can create a high speed and low power Viterbi encoder decoder at the same time with some extra hardware area.

I. INTRODUCTION

CONVOLUTIONAL (Viterbi) decoding is a Forward Error Correction (FEC) technique [1]-[2]. The purpose of FEC is to improve the capacity of a channel by adding some carefully designed redundant information to the data being transmitted through the channel. The process of adding this redundant information is known as channel coding. Convolutional coding and block coding are the two major forms of channel coding. Convolutional codes operate on serial data, one or a few bits at a time. Block codes operate on relatively large (typically, up to a couple of hundred bytes) message blocks. There are a variety of useful convolutional and block codes, and a variety of algorithms for decoding the received coded information sequences to recover the original data.

In this paper, a design of high speed low power Viterbi decoder at the RTL level in the standard cell design environment is proposed. In the standard cell design environment, the behavior of a design is described in VHDL. The behavioral design is synthesized to generate a gate level design. The gate-level design is placed and routed to generate a layout of the design. The advantages of a standard cell based design over full custom design are faster turnaround time for the design, ease in design verification and more accurate modeling of the circuit. Design of Viterbi decoders at the RTL-level is focused here. Viterbi algorithms and implementation of Viterbi decoders were investigated intensively in the past three decades [3]-[5].

Pooran Singh is with VLSI/ULSI Circuit & System Design Lab, School of Engineering, Electrical Engineering Discipline, Indian Institute of Technology (IIT)Indore, M.P., India-453 441 phd11120203@iiti.ac.in

Kr. Vishvakarma is with VLSI/ULSI Circuit & System Design Lab, School of Engineering, Electrical Engineering Discipline, Indian Institute of Technology (IIT) .skvisvakarma@iiti.ac.in.

This paper is organized as follows. Viterbi encoder of length $k = 3$ is describes in Section 2. Section 3 describes the design for Viterbi decoders. Section 4 shows the power and speed in various FPGA kits and also defines the area in terms of logic utilization. Section 5 summarizes the paper.

II. VITERBI ENCODER

A. Overview And Introduction

First, convolutional codes are discussed. Then, we explain the Viterbi decoding algorithm and the Viterbi decoder design. Fig. 1 below shows the Viterbi (convolutional) encoder with constraint length of 5.

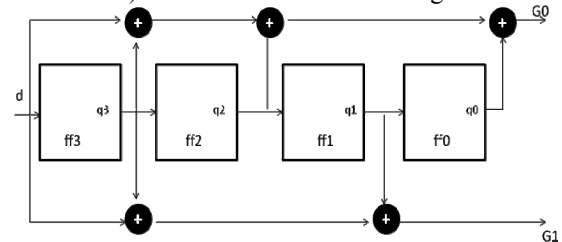


Fig. 1. Viterbi encoder (convolutional encoder) with $k = 5$.

B. Convolutional Codes

The convolutional encoder can be viewed as a finite state machine. It generates a coded output data stream from an input data stream. It is usually composed of shift registers and a network of XOR (Exclusive-OR) gates as shown in Fig. 1. The encoder in Fig. 1 produces two bits of encoded information for each bit of input information, so it is called a rate $1/2$ encoder. A convolutional encoder is generally characterized in (n, k, m) format, where n is number of outputs of the encoder; k is number of inputs of the encoder; m is number of memory elements (flip-flops) of the longest shift register of the encoder.

The rate of a (n, k, m) encoder is k/n . The encoder shown in the figure is a $(2, 1, 4)$ encoder with rate $1/2$. In this paper, we discuss decoding of convolutional codes generated by a $(2, 1, 4)$ encoder with the rate $\frac{1}{2}$ as in Fig. 2.

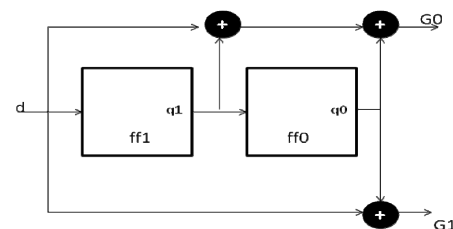


Fig. 2. Encoder for rate $\frac{1}{2}$ and $k = 3$.

Fig. 3 represents the state table of the encoder. It depicts state transitions and the Fig. 4 shows the corresponding encoded outputs. As there are two memory-elements in the circuit, there are four possible states that the circuit can assume.

Current State	Next State, if	
	Input = 0:	Input = 1:
00	00	10
01	00	10
10	01	11
11	01	11

Fig. 3. State table of encoder k=3.

Current State	Output Symbols, if	
	Input = 0:	Input = 1:
00	00	11
01	11	00
10	10	01
11	01	10

Fig. 4. Output state table of encoder.

These four states are represented as S0 through S3. Each state's information (i.e. the contents of flip-flops for the state) along with an input generates an encoded output code. For each state, there can be two outgoing transitions; one corresponding to a '0' input bit and the other corresponding to a '1' input bit. A trellis diagram is an extension of a state diagram that explicitly shows the passage of time.

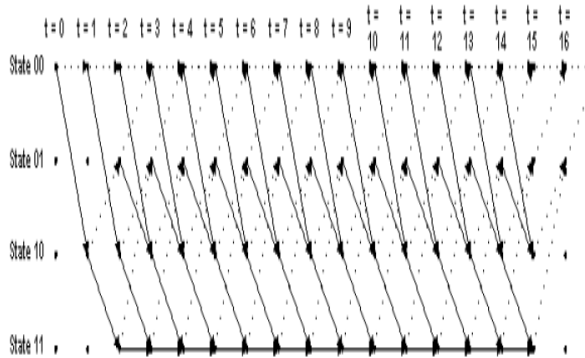


Fig. 5. Trellis diagram

Fig. 5 shows a trellis diagram for the encoder given in Fig. 2. In the trellis diagram, nodes correspond to the states of the encoder. From an initial state (S0) the trellis records the possible transitions to the next states for each possible input pattern. For the encoder in Fig. 2, there are two encoded symbols corresponding to input bit '0' and '1'. The Fig. 5 shows the encoded symbol generated for each transition. At the stage t=1 there are two states S0 and S1, and each state has two transitions corresponding

to input bits '0' and '1'. Hence the trellis grows up to the maximum number of states or nodes, which is decided by the number of memory elements in the encoder.

III. IMPLEMENTATION OF VITERBI DECODER

The major tasks in the Viterbi decoding process are as follows:

- 1) *Branch metric computation.*
- 2) *State metric update:* Update the state metric using the new branch metric.
- 3) *Survivor path recording:* Tag the surviving path at each node.
- 4) *Output decision generation:* Generation of the decoded output sequence based on the survivor path information.

Fig. 6 shows the proposed Viterbi decoder. This section discusses the different parts of the Viterbi decoding process. Analog signals are quantized and converted into digital signals in the quantization block. The synchronization block detects the frame boundaries of code words and symbol boundaries. We assume that a Viterbi decoder receives parallel successive code symbols, in which the boundaries of the symbols and the frames have been identified.

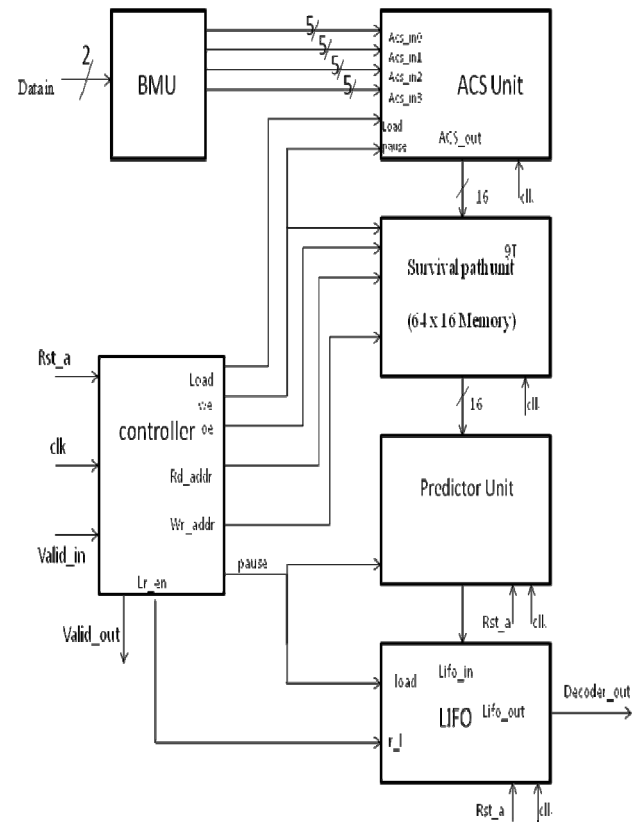


Fig. 6. Internal sub blocks of proposed viterbi decoder.

C. Branch Matrix Unit

It is used to generate branch metrics, which are hamming distances of input data from 00, 01, 10 and 11. The BM unit is used to calculate branch metric for all trellis branches from the input data. We choose absolute difference as measure for branch metric. These branch

metrics are viewed as being the weights of the branches. The block diagram of BMU is shown in Fig. 7.

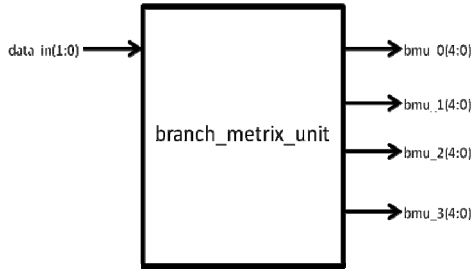


Fig. 7. Block diagram of branch matrix unit.

D. ACS (Add Compare Select) Unit

A new value of the state metrics has to be computed at each time instant. In other words, the state metrics have to be updated every clock cycle. Because of this recursion, pipelining, a common approach to increase the throughput of the system, is not applicable. The Add-Compare-Select (ACS) unit hence is the module that consumes the most power and area. Fig. 8 shows the internal block diagram of single ACS sub block.

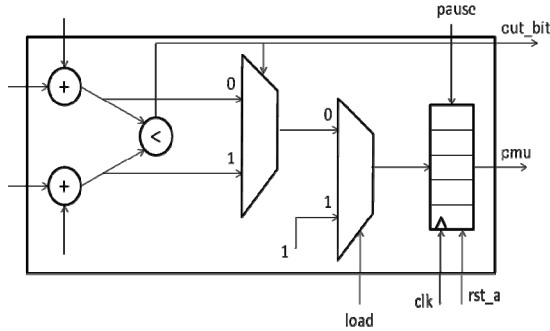


Fig. 8. Internal diagram of a single ACS sub block.

In order to obtain the required precision, a resolution of 5 bits for the state metrics is essential, while 5 bits are needed for the branch metrics. Since the state metrics are always positive numbers and since only positive branch metrics are added to them, the accumulated metrics would grow indefinitely without normalization. The operation of the ACS unit is shown in Fig. 9.

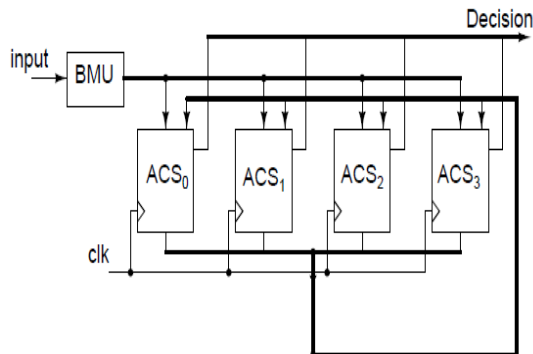


Fig. 9. ACS unit.

The new branch metrics are added to previous state metrics to form the candidates for the new state metrics. The comparison can be done by using the subtraction of the two candidate state metrics, and the MSB of the difference points to a larger one of two.

E. Memory

Memory is required to store the survivor Path Matrix Unit (PMU) [8]. The word length of the memory depends on the number of the ACS sub-blocks used in the design or the total number of states in the decoder or k^2 (where k is the constraint length, 5 in our case), and the depth of the memory depends on the trellis length. The memory depth usually should be kept two times the trellis length or two blocks of memory equal to trellis length. We have for our project $k = 5$ and trellis length equal to 32, so the memory block used is 64×16 . The memory used is dual port. One port for writing the data and other for reading the data, as we need to write and read the data simultaneously and that to from different addresses. Memory should write data synchronously but the reading of the data should be asynchronous to keep the latency low or better manage the synchronous behavior of the full system.

F. Controller

A controller is used to synchronize between the different modules of the system. The controller unit of decoder controls signals like *we*, *pause*, *valid_out*, *oe*, *rd_addr*, *wr_addr*.

The controller also includes two six bit counters of which one counts up another counts down. These counters drive the write and read addresses of the memory. The pause signal generated by the controller also stops these counters for a while so that no unnecessary data is written onto the memory or read from it.

G. Predictor Unit

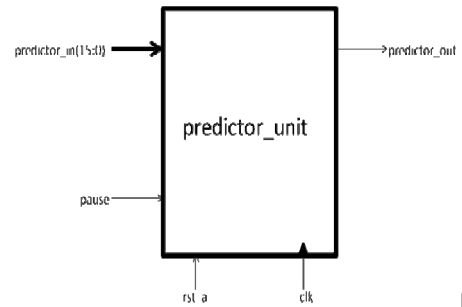


Fig. 10. Block diagram of predictor unit.

Predictor unit is used to trace back the trellis sequence of length 32 and predict the next state and actual decoded bit after rectifying the error. This unit is a state machine that is loaded with the state with minimum accumulated path metric after every 32 clock cycles. This unit uses the state value to access a bit from the path metric memory unit (PMM) or memory unit. Block diagram of predictor unit is shown Figure 10. Functional description of predictor unit is in Table 1.

TABLE I
FUNCTIONAL DESCRIPTION OF REDICTOR UNIT.

If rst_a is high	Fsm become at state s0(initial state)
If pause is high	Fsm remain in the same state and output remain in the same value
Other conditions	State transition occurs in fsm and generate the output

Predictor unit consist of 16:1 multiplexer and a state machine which is used to generate as well as rectify the error. Each present state of state machine has only two possible next states. According to the present state of the state machine, respective multiplexer input is selected and then check that bit, if the bit is 0 then the smaller possible state is selected and if the bit 1 then bigger state is selected. Based on the present state and next state data, actual decoded output is generated which is then send to the LIFO input.

H. LIFO Unit

Every 32 decoded bits put out by the predictor unit is in reverse order of the transmitted data, this necessitates a LIFO unit. This unit has 2 32-bit registers in which one of the register is read while the other is written. The two 32-bit registers are read and write alternatively and simultaneously selected by the multiplexer which is in the read mode and gives the output in correct sequence. The block diagram of LIFO is shown in Fig.11. Functional description of LIFO unit is shown in Table 2.

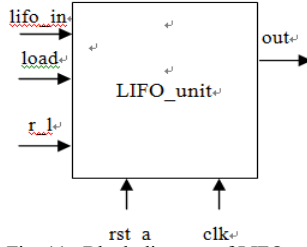


Fig. 11. Block diagram of LIFO unit.

TABLE II
FUNCTIONAL DESCRIPTION OF LIFO UNIT

If rst_a is high	Both 32-bit registers become reset
If load is high	Read and write the data from the registers
If load is low	Neither read nor write the data
If r_l is high	Upper register in read mode and lower in write mode
If r_l is low	Upper register in write mode and lower in read mode

IV. RESULTS & COMPARISONS

The implementation of Viterbi encoder decoder with K=3 is done. Our work is based on about to make a Viterbi Encoder- Decoder fast with low power consumption. So for getting high speed we used Virtex 7 Kit to generate a 393.544 MHz clock frequency of our proposed encoder-decoder and for low power optimization we used Spartan 6 Low voltage Xilinx FPGA [9] which shows a power reduction of Viterbi decoder upto 11.34 mW. The results are calculated in Xilinx synthesis tool and the power optimization is done in X-power Estimator in Xilinx. The results shows that there is about 11.34 mW of Dynamic Power consumed by the logic and it run with 393.544 MHz Clock frequency.

A. Synthesis Report

In following synthesis report, Table III shows the logic utilization and Table IV shows the clock information while Table V shows as the minimum power consumption of Spartan 6 low voltage FPGA kit.

a) Device used for high speed of 393.544 MHz

Target Device: Vertex7, Family: XC7VX330T, Package: FFG1157, Speed: -3.

TABLE III
LOGIC UTILIZATION INFORMATION.

Slice Logic	No. of Logic used
No. of Slice Registers	198 out of 408000
No. of Slice LUTs	390 out of 204000
No. used as Logic	370 out of 204000
No. used as Memory	22 out of 70200
No. used as RAM	22
Number of 6 input LUT Flip Flop pairs used	394

TABLE IV
CLOCK INFORMATION

Maximum Frequency	393.544 MHz
Setup time	2.34ns
Hold time	1.57ns

b) Device used for 11.34 mW low power.

Target Device: Spartan 6 low Power, Family: XC6SLX4L, Package: TQG144, Speed: -1L

TABLE V
POWER SUPPLY SUMMARY

Supply Power (mW)	Dynamic	Quiescent	Total
	10.97	0.37	11.34

TABLE VI
COMPARISONS WITH OTHER WORKS

Terms	Proposed work	Jinjin He et al. [10]	Xin-Ru Lee et al. [11]	Kelvin Yi-Tse Lai [12]
Power(mw)				
Power (mW)	11.34 (Decoder)	21.47 3	2.48	63.97
Clock Frequency (MHz)	413 (Decoder), 393.544 (Encoder-Decoder)	505 450	278 230	250 228
Area (mm ²)	0.98 (Decoder only)	0.58	2.3	0.121
Logic Utilization of FPGA (% of Slices used)	2% (For Both)	1%	3%	1%

Table VI shows the various comparisons of viterbi decoder and encoder-decoder at K=3, with other related works and it showing that our work is maintains good speed with a cost of low power as well. Jinhin He et al. [10] viterbi works at higher frequency clock and Xin-Ru Lee et al. [11] at lower power but out proposed work carry both power and speed and make it highly optimized.

V. CONCLUSION

Hence from our proposed Viterbi encoder decoder we designed a high speed of 393.544 MHz clock and low power with 11.34 mW total power consumption decoder and which can be used for communication protocols like CDMA for high speed data transmission with nearly low power with less amount extra logic utilization.

REFERENCES

- [1] Tomas, V., "Decoding of Convolutional Codes Over the Erasure Channel," *IEEE Trans. on Information Theory*, vol.58, no.1, pp. 90-108, Jan. 2012.
- [2] Viterbi AJ., "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, Vol. 13, no. 2, pp. 260–269, April 1967.
- [3] Bupesh Pandita and Subir K Roy., "Design and Implementation of Viterbi Decoder Using FPGAs," *VLSID '99 Proceedings of the 12th International Conference on VLSI Design*, page 611, 1999.
- [4] P.J. Black and T.H. Meng., "A 1-Gb/s, four-state, sliding block Viterbi decoder," *IEEE Journal of Solid-State Circuits*, Vol. 32 no.6, pp.797–805, 1997.
- [5] M. Boo, F. Arguello, JD Bruguera, R. Doallo, and EL Zapata., "High-performance VLSI architecture for the Viterbi algorithm," *IEEE Trans. on communications*, Vol. 45, no.2 pp.168–176, 1997.
- [6] O. Collins and F. Pollara., "Memory management in traceback Viterbi decoders," *TDA Prog. Rep*, pages 42–99, 1989.
- [7] G. Fettweis and H. Meyr., "Parallel Viterbi decoding by breaking the compare select feedback bottleneck," *Communications*, 201:88, 1988.
- [8] G. Feygin and P. Gulak., "Architectural tradeoffs for survivor sequence memory management in Viterbi decoders," *IEEE Transactions on Communications*, Vol. 41, no.3, pp. 425– 429, 1993.
- [9] Xilinx, "Lowering power at 28 nm with Xilinx 7 Series FPGAs," White paper, WP389 (v1.1.1), 2012.
- [10] Jinjin He et al., "High-Speed Low-Power Viterbi Decoder Design for TCM Decoders," *IEEE Trans. on Very Large Scale Integration (VLSI) Systems*, Vol. 20, No. 4, April, 2012.
- [11] Xin-Ru Lee et al., "A Low-Power Radix-4 Viterbi Decoder Based on DCVSPG Pulsed Latch with Sharing Technique," *IEEE Asia Pacific conference on Circuits & systems (APCCS-2010)*.
- [12] Kelvin Yi-Tse Lai, "An Efficient Metric Normalization Architecture for High-speed Low- power Viterbi Decoder," *TENCON-2007, IEEE Region Ten Conference*.